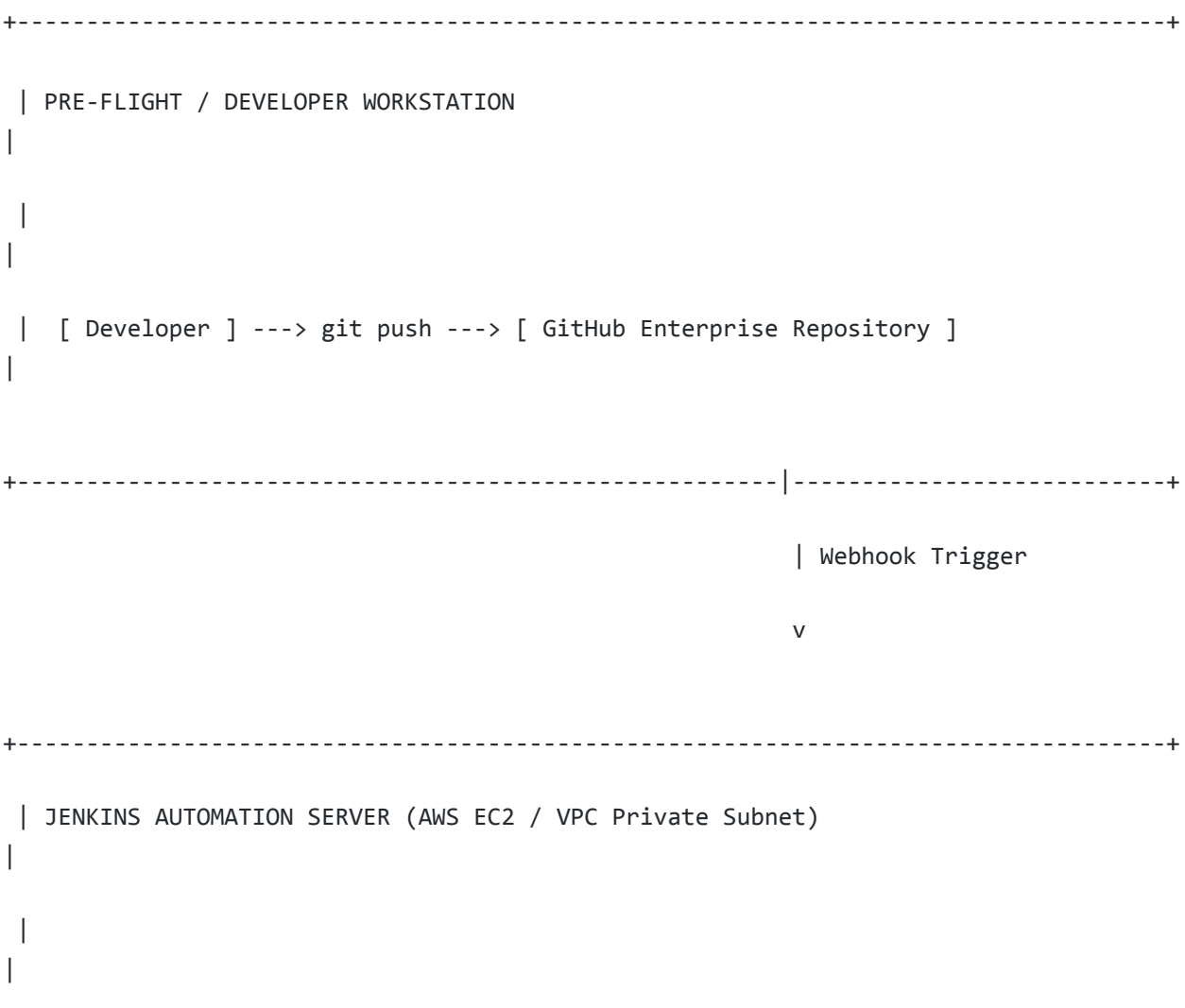
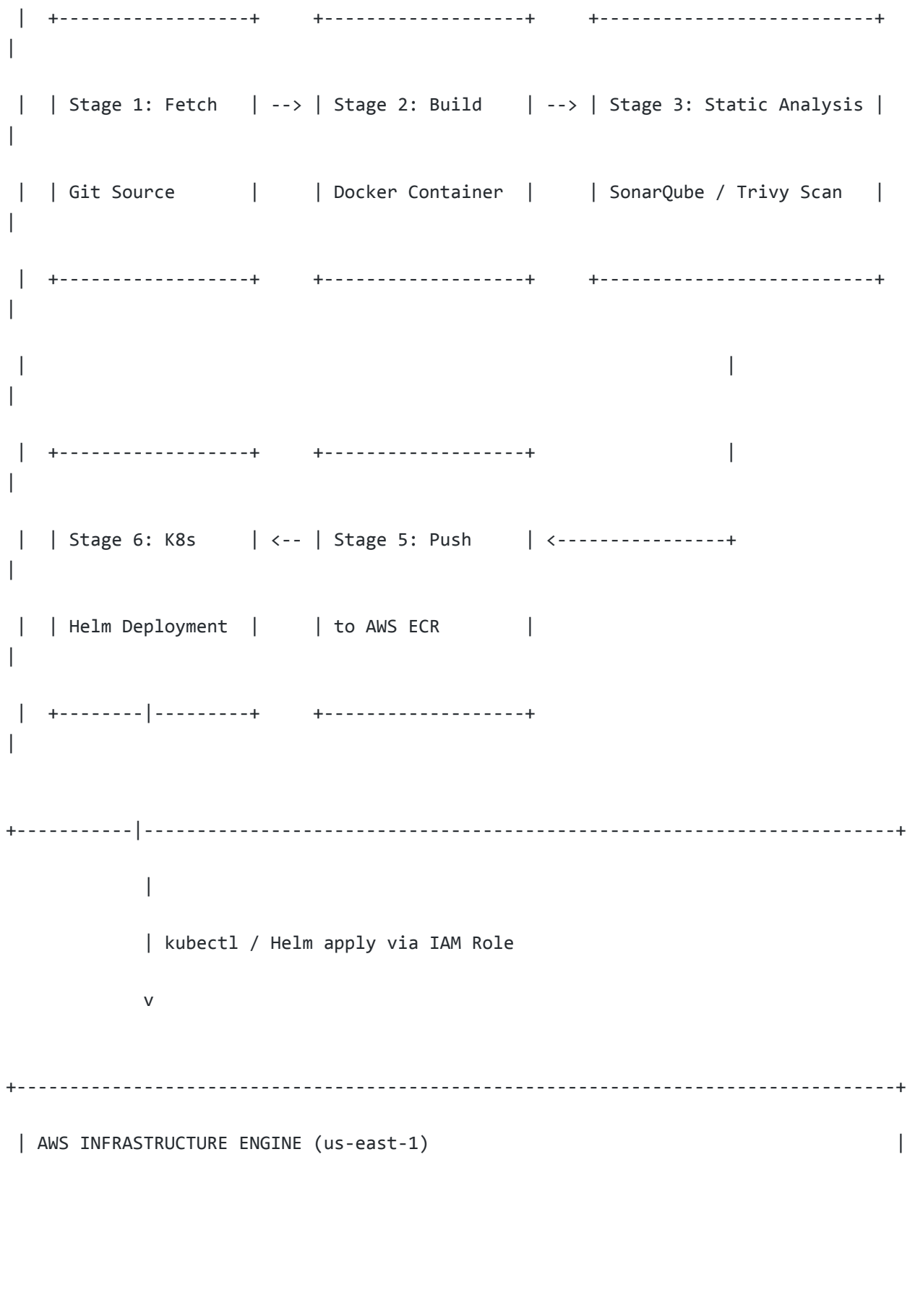
Automated CI/CD Infrastructure Blueprint

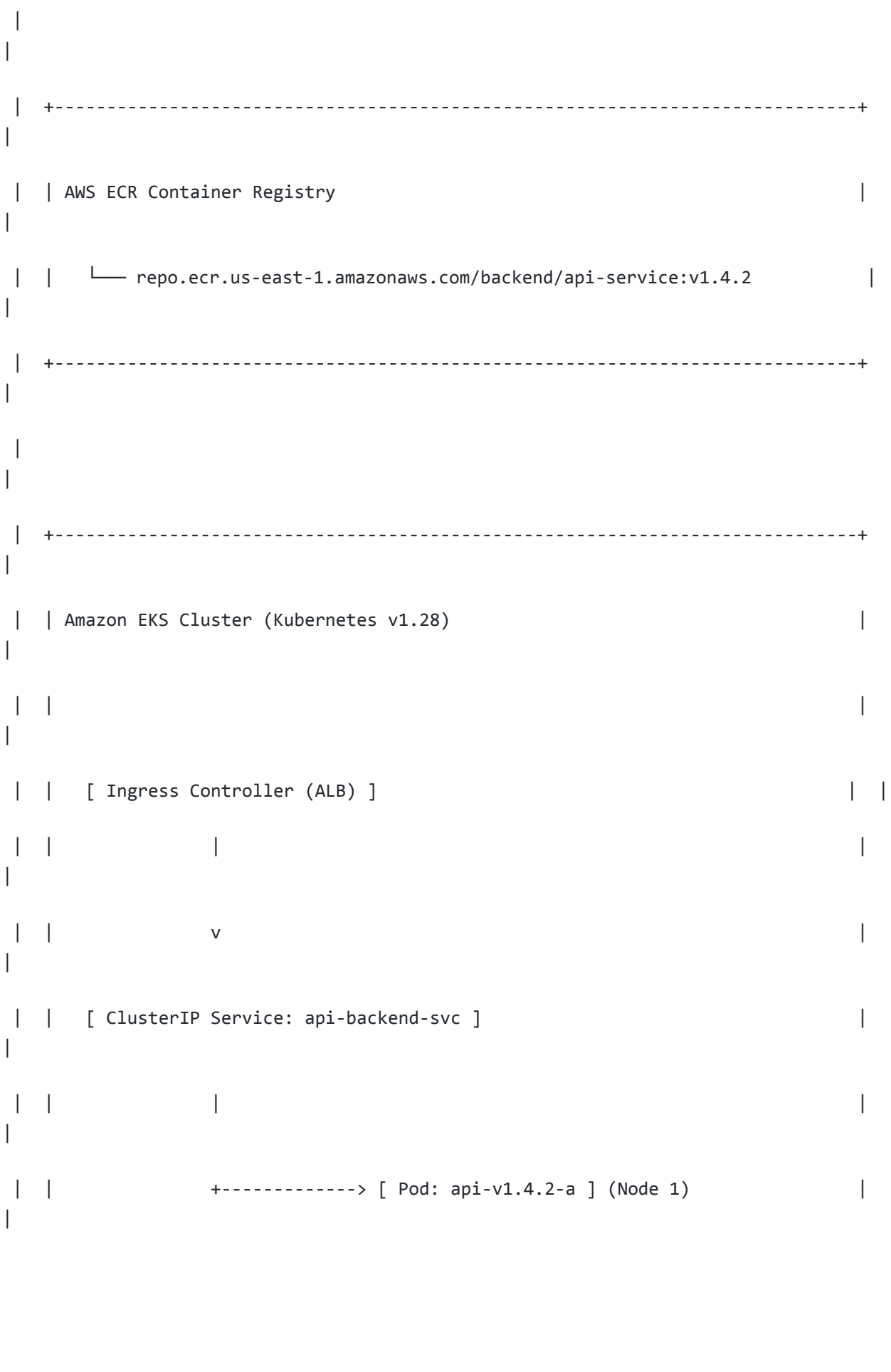
Continuous Integration, Container Orchestration, and AWS Target State Deployment

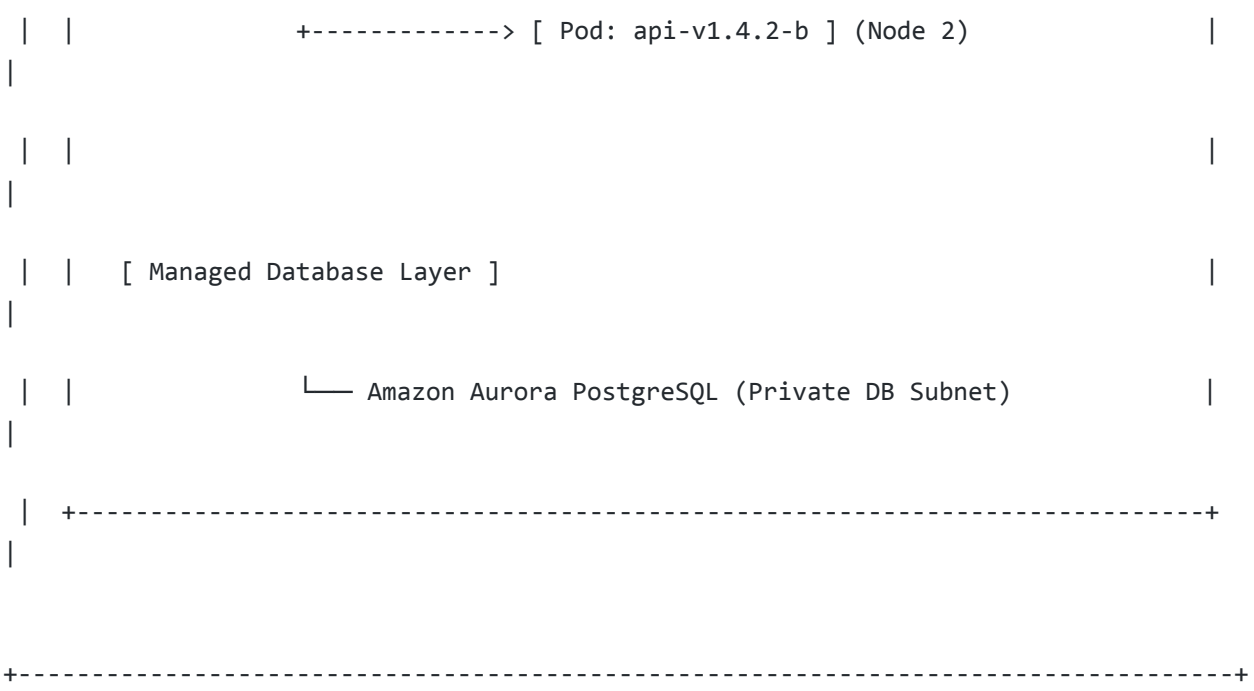
This document outlines the technical specification for the automated CI/CD pipeline powering the core backend service infrastructure. The pipeline utilizes Jenkins for execution, Docker for containerization, Amazon ECR for artifact storage, and Amazon EKS (Kubernetes) for runtime orchestration.

1. System Architecture Diagram




Stage 1: Fetch
Git Source
Stage 2: Build
Docker Container
Stage 3: Static Analysis
SonarQube / Trivy Scan
Stage 6: K8s
Helm Deployment
Stage 5: Push
to AWS ECR
kubectl / Helm apply via IAM Role
AWS INFRASTRUCTURE ENGINE (us-east-1)


AWS ECR Container Registry
repo.ecr.us-east-1.amazonaws.com/backend/api-service:v1.4.2
Amazon EKS Cluster (Kubernetes v1.28)
[Ingress Controller (ALB)]
[ClusterIP Service: api-backend-svc]
[Pod: api-v1.4.2-a] (Node 1)



2. Pipeline Execution Stages

Stage 1: Checkout & Environment Initialization

- Event-driven trigger execution via GitHub webhook (push event to main or release branches).
- Initialization of ephemeral Jenkins agent pod operating within the AWS EKS build namespace.
- Authentication credentials loaded dynamically via AWS IAM Roles for Service Accounts (IRSA).

Stage 2: Container Image Construction

- Build executed using Docker CLI (docker buildx) utilizing multi-stage builds to ensure minimal image payload footprint.
- Dockerfile execution optimizes layer caching by separating dependency manifests from source code compilation.

Stage 3: Static Analysis & Vulnerability Scanning

- Static Code Analysis (SCA) via SonarQube scanner for quality gate enforcement.
- Container security analysis via Trivy scanner executing image CVE evaluation prior to registry push. Pipeline halts if critical CVE threshold is breached.

Stage 4: Artifact Tagging & Registry Push

- Image tagging follows Semantic Versioning along with the short Git commit SHA: 1.4.2-b8d9e1f.
- Authentication against AWS ECR using temporary token provided via aws ecr get-login-password.
- Image pushed to high-availability target Amazon ECR repository.

Stage 5: Kubernetes Manifest Interpolation & Deployment

- Helm configuration values dynamically rendered per environment target (Staging vs. Production).
- Zero-downtime Rolling Update deployment strategy executed via helm upgrade --install.
- Automatic readiness probe checking ensures active traffic routing occurs only after container health validation (/healthz endpoint returning HTTP 200).

3. Environment Connection Configurations

Below are reference connection strings for runtime microservices and administrative tooling within the isolated VPC network segments.

Database Connection Strings

- Primary Production Read-Write Cluster: postgresql://db_master_admin:a8f9c2d7e1b30495a@aurora-pg-prod-cluster.

cluster-c92x81mks.us-east-1.rds.amazonaws.com:5432/core_backend_db?sslmode=verify-full&sslrootcert=/etc/ssl/certs/rds-ca-2021.pem

- Read-Replica Pool (Reporting Services): postgresql://db_read_only:f3b2018a471e9c20d@aurora-pg-prod-ro.cluster-c92x81mks.us-east-1.rds.amazonaws.com:5432/core_backend_db?sslmode=verify-full&target_session_attrs=read-only

Caching Layer Connection Strings

- Elasticache Redis Cluster (TLS Enabled): rediss://:eY3xP9vL1qR8zW5mK2nT7sB0aC4vF6gH@prod-redis-cluster.cache.us-east-1.amazonaws.com:6379/0

Queue & Message Broker Connection Strings

- Amazon MQ (ActiveMQ) Broker Connection: ssl://b-4910283f-8a02-41e9-918d-3298a0d018a2-1.mq.us-east-1.amazonaws.com:61616?wireFormat.maxFrameSize=104857600

Internal Kubernetes Cluster Access

- EKS API Server Endpoint: https://B8E301F29C400A2D8197110C1A39D9B2.gr7.us-east-1.eks.amazonaws.com
- Internal Service Mesh Ingress Endpoint: https://internal-alb-ingress-prod-1029384756.us-east-1.elb.amazonaws.com/api/v1

4. Pipeline Infrastructure Specifications

Component	Provider / Tool	Technical Configuration	Scaling / Resiliency Policy
Source Control	GitHub Enterprise	Branch protection on main, mandatory PR reviews	Multi-region redundant hosting
Automation Server	Jenkins v2.414	EC2 Deployment, Docker-in-Docker (dind) execution agents	Dynamic Auto Scaling Group (1-10 instances)
Container Runtime	Docker v24.0	Multi-stage alpine-based build containers	Local registry cache enabled
Artifact Registry	Amazon ECR	Private repository, AES-256 encryption at rest	Multi-AZ replication, lifecycle rules (30 days)
Orchestrator	Amazon EKS v1.28	Managed Node Groups (m5.xlarge), Cilium CNI	Auto-scaling groups spanning 3 Availability Zones
Ingress Control	AWS ALB Ingress	AWS Load Balancer Controller with ACM TLS Termination	Cross-zone load balancing with health checks